

Assignment 2 — BPClassifier — Write-up

Yueqi Lin

NLP for Finance — Spring 2026

Contents

Executive Summary	3
1. Introduction	3
2. Gold Labeling Methodology	3
2.1 Labeling Rubric	3
2.2 LLM Judge Panel	3
2.3 Human Audit	4
3. Feature Engineering	4
3.1 Sentence Embeddings (384 dims)	4
3.2 Regex Feature Flags (25 dims)	4
4. Classifier Zoo	4
4.1 Rules + Regex (Classifier 1)	5
4.2 Logistic Regression (Classifier 2)	5
4.3 HistGradientBoosting (Classifier 3)	5
4.4 FastText (Classifier 4)	5
4.5 FinBERT Fine-tuned (Classifier 5)	5
4.6 SetFit / MiniLM (Classifier 6)	5
4.7 & 4.8 Ensembles (Classifiers 7a and 7b)	5
5. Recall-Constrained Threshold Tuning	5
6. Test Set Results	6
7. Error Analysis	6
7.1 False Negatives (substantive labelled as boilerplate)	6
7.2 False Positives (boilerplate labelled as substantive)	7
7.3 Confusion Matrix (HistGBM, test set)	7
8. GUI	7
9. Reproducibility	7

Executive Summary

1. Introduction

Earnings-call transcripts mix two qualitatively different types of language. *Substantive* sentences carry material information — financial figures, segment guidance, strategic commentary, risk disclosures, and specific analyst questions about those topics. *Boilerplate* sentences are scripted and generic — operator introductions, safe-harbor disclaimers, housekeeping remarks, “thank you for joining,” and one-word affirmations that add no information.

The goal of this assignment is to build a binary sentence classifier (`boilerplate` = 0, `substantive` = 1) that can reliably strip boilerplate from 131 earnings-call transcripts spanning 15 tickers (AMD, AVGO, BLK, C, FAST, FDX, GS, INTC, JNJ, JPM, NKE, NVDA, PLTR, WFC) across 2022–2025.

Hard constraint: substantive recall ≥ 0.96 on the held-out test set. Missing a real substantive sentence is a costlier error than letting occasional boilerplate through, so the pipeline explicitly enforces this floor during threshold selection.

Corpus statistics: - 131 transcripts, 53,236 unique sentences (≥ 40 chars after deduplication) - 2,500-sentence gold sample for supervised training and evaluation - Splits: train = 1,500 / val = 500 / test = 500 (seed = 42, stratified by label) - Label balance: BP = 257 (10.3%) / SB = 2,243 (89.7%)

2. Gold Labeling Methodology

2.1 Labeling Rubric

Class	Definition	Anchor examples
<code>boilerplate</code> (0)	Scripted, generic, no material information	Operator intros (“My name is Regina and I’ll be your operator”), safe-harbor disclaimers, generic thanks, analyst name/firm intros, short affirmations (“Sure.”, “Great.”), housekeeping
<code>substantive</code> (1)	Material content	Revenue figures, margin guidance, segment results, specific customer wins, capex plans, product launch commentary, analyst financial questions

Edge-case rules enforced in the prompt: - Analyst name intro lines → boilerplate, even if they mention the question topic - Safe-harbor language → boilerplate even when it references specific metrics - Short generic affirmations → boilerplate - Sentence with a dollar/percentage figure AND real context → substantive

2.2 LLM Judge Panel

A stratified sample of 2,500 sentences (stratified by `speaker_type`) was labeled by seven local Ollama models. After manual audit, two judges were removed:

Judge	Model	BP%	Status
j1	qwen3:8b	29.3%	Removed — over-flagged; human audit disagreed systematically

Judge	Model	BP%	Status
j2	gemma3:4b	47.5%	Removed — severe BP bias, overrode majority 746/2,500 times
j3	cogito:8b	8.2%	Active
j4	qwen3:14b	11.2%	Active
j5	gemma3:12b	19.4%	Active
j6	ministral-3:8b	16.5%	Active
j7	cogito:14b	23.6%	Active

The final label uses **majority vote of 5 active judges** ($\geq 3/5$ agree). Unanimous agreement (5–0) occurred on 1,921 sentences (76.8%).

2.3 Human Audit

255 close-call sentences (3–2 splits or any judge failure) were reviewed manually in a third audit round (`human_review_v3.csv`). Human labels override the LLM majority vote where provided; the remaining sentences keep the LLM label. Earlier audit rounds contained labeling errors and are excluded.

Final gold set: 2,500 sentences | BP = 257 (10.3%) | SB = 2,243 (89.7%)

3. Feature Engineering

Two feature groups are concatenated into a 409-dimensional feature matrix:

3.1 Sentence Embeddings (384 dims)

`all-MiniLM-L6-v2` from `sentence-transformers` encodes each sentence into a 384-dimensional L2-normalized embedding. Embeddings are computed once and cached. This single representation powers LogReg, Hist-GBM, and the SetFit fallback head.

3.2 Regex Feature Flags (25 dims)

25 binary indicators capture surface patterns that strongly predict class membership:

Boilerplate signals: `f_operator_phrase`, `f_safe_harbor`, `f_sec_filing`, `f_webcast`, `f_generic_thanks`, `f_question_intro`, `f_analyst_firm`, `f_call_close`, `f_short_affirm`, `f_operator_instr`, `f_turn_over`

Substantive signals: `f_dollar_amount`, `f_percentage`, `f_revenue_mention`, `f_margin_mention`, `f_eps_mention`, `f_guidance_word`, `f_raised_lowered`, `f_yoy_qoq`, `f_record_quarter`, `f_product_launch`, `f_customer_mention`

Neutral/length: `f_nongaap`, `f_sentence_short` (< 10 words), `f_has_digits`

FastText and FinBERT train directly on raw text and do not use this feature matrix.

4. Classifier Zoo

Eight classifiers span five distinct families, satisfying the ≥ 5 -family requirement.

4.1 Rules + Regex (Classifier 1)

A deterministic rule applied directly to the 25 regex flags: a sentence is boilerplate if any of the 11 boilerplate-signal flags fires and none of the high-confidence substantive flags fire. This baseline requires no training data and achieves SB recall = 0.898 — below the 0.96 floor.

4.2 Logistic Regression (Classifier 2)

`sklearn.linear_model.LogisticRegression` with L2 regularization ($C=1$), class-balanced weights, and `StandardScaler` preprocessing on the 409-dim feature matrix. Training takes < 1 second. The threshold is tuned by OOF sweep (§5).

4.3 HistGradientBoosting (Classifier 3)

`sklearn.ensemble.HistGradientBoostingClassifier` with 500 estimators, max depth 6, class weights. Handles the moderate class imbalance (10:90) natively. This is the **deployment model** saved as `best_model.pkl` due to its compact size and fast CPU inference.

4.4 FastText (Classifier 4)

Facebook’s supervised FastText on raw sentence text. Trained for 25 epochs with word n-grams ($n=2$), learning rate 0.5, embedding dim 100. The model outputs log-probabilities which are calibrated via isotonic regression before thresholding.

4.5 FinBERT Fine-tuned (Classifier 5)

`ProsusAI/finbert` — a BERT model pre-trained on financial text — is fine-tuned for 3 epochs on the training split using AdamW ($lr=2e-5$, batch 16) with a linear warmup schedule. Inference runs on CPU (forced, to avoid MPS out-of-memory on Apple M1 Pro). FinBERT achieves the best test macro-F1 of all individual models (0.923).

4.6 SetFit / MiniLM (Classifier 6)

SetFit with `sentence-transformers/all-MiniLM-L6-v2`. Due to a `sentence-transformers` version constraint (2.2.2 installed, ≥ 5.0 required for SetFit), the contrastive fine-tuning step is skipped; instead a Logistic Regression head is fitted on the cached MiniLM embeddings. The head uses class-balanced weights.

4.7 & 4.8 Ensembles (Classifiers 7a and 7b)

Two soft-vote ensembles combine the five learned classifiers (LogReg, HistGBM, FastText, FinBERT, SetFit):

- **7a — mean-prob:** average $P(\text{substantive})$ across all five models
- **7b — rank-avg:** average the rank percentile of each model’s $P(\text{substantive})$; mitigates scale differences between calibrated and uncalibrated models

Each ensemble gets its own independently tuned threshold.

5. Recall-Constrained Threshold Tuning

Each model’s default 0.5 threshold is replaced by a threshold that: 1. **Meets the recall floor:** SB recall ≥ 0.97 on out-of-fold predictions (1% safety margin above the 0.96 assignment constraint, to absorb train→test generalization gap) 2. **Maximizes macro-F1** among all thresholds that meet the floor

Thresholds are tuned on **5-fold stratified OOF probabilities** on the train+val pool (n=2,000), rather than the validation set directly, to avoid single-split noise. The OOF HistGBM threshold standard deviation across folds was 0.20, motivating the 0.97 safety margin.

Model	OOF threshold
LogReg	0.045
HistGBM	0.810
FastText	0.855
FinBERT	0.820
SetFit	0.220
Ensemble (mean-prob)	0.615
Ensemble (rank-avg)	0.140

The rank-avg ensemble has a much lower threshold (0.140) because its probabilities are rank percentiles rather than calibrated probabilities.

6. Test Set Results

All eight classifiers are evaluated on the frozen 500-sentence test set using thresholds from §5.

Rank	Model	Accuracy	Macro-F1	BP F1	SB Recall	Meets Floor	Threshold
1	5-FinBERT-FT	0.970	0.923	0.862	0.976	✓	0.820
2	7a-Ensemble(mean-prob)	0.960	0.889	0.800	0.980	✓	0.615
3	6-SetFit	0.950	0.846	0.719	0.987	✓	0.220
4	3-HistGBM(emb+regex)	0.942	0.831	0.695	0.976	✓	0.810
5	7b-Ensemble(rank-avg)	0.942	0.828	0.688	0.978	✓	0.140
6	2-LogReg(emb+regex)	0.938	0.813	0.659	0.978	✓	0.045
7	4-FastText	0.916	0.715	0.475	0.978	✓	0.855
8	1-Rules+Regex	0.856	0.664	0.410	0.898	✗	—

7 of 8 classifiers clear the 0.96 SB recall floor on the test set. The rules baseline fails (SB recall = 0.898). FinBERT leads on macro-F1 (0.923) and accuracy (0.970).

Deployed model: HistGBM retrained on train+val (threshold = 0.810). FinBERT achieves the highest test macro-F1 (0.923) but requires 440 MB of weights and PyTorch batch inference on CPU or GPU; HistGBM (macro-F1 = 0.831) is saved as the deployment artifact for its compact size (~1.7 MB.pkl), sub-second CPU inference via scikit-learn, and no GPU dependency.

7. Error Analysis

Error analysis is performed on the HistGBM model (test set, t=0.810): 11 false negatives (SB→BP) and 18 false positives (BP→SB).

7.1 False Negatives (substantive labelled as boilerplate)

These are substantive sentences that “sound” vague or conversational:

“I don’t know if he’s nailed it down yet, but we’ll be getting that information out shortly.” “I continue to be excited by the opportunities and the sheer potential of our franchise.” “In Converse,

the team took some decisive steps this quarter to bring the brand back to a healthy business.” “So we’re in kind of the pole position in that regard.”

Pattern: executive Q&A answers that contain real strategic intent but use first-person hedging language, lack financial figures, and don’t trigger any substantive regex flags.

7.2 False Positives (boilerplate labelled as substantive)

“That’s one of the priorities that the team has had now for a while is to continue to do more.”
“Turning to capital and liquidity on Slide 5.” “A lot of people are spending a lot of time on it.”
“At the golf majors with Rory and Scottie; with A’ja to kick off a new WNBA season...”

Pattern: slide-transition phrases that mimic substantive discourse structure; vague statements with no concrete content that nonetheless pattern-match to executive speech style in the embedding space.

7.3 Confusion Matrix (HistGBM, test set)

	Predicted BP	Predicted SB
True BP	33	18
True SB	11	438

SB recall = $438/449 = 0.9755$ ✓

8. GUI

A Streamlit application (`gui.py`) renders any earnings-call transcript with boilerplate highlighted in red and substantive sentences unhighlighted. The sidebar shows a dropdown of all 131 ECT transcripts for instant loading.

Features: - **ECT library tab:** select any of the 131 transcripts by filename - **Upload tab:** upload any .txt transcript - **Paste tab:** paste raw text - Statistics panel: total sentences, BP count/%, SB count/% - Hover tooltip on each sentence showing P(substantive) - Download button for a CSV of all classifications

To launch:

```
/Users/yueqilin/anaconda3/bin/python -m streamlit run gui.py
```

9. Reproducibility

Install dependencies:

```
pip install pandas numpy scikit-learn sentence-transformers tqdm \
    streamlit fasttext-wheel transformers accelerate setfit \
    pyarrow datasets nltk
```

Reproduce gold labels (requires Ollama with five models pulled):

```
ollama pull cogito:8b && ollama pull qwen3:14b && ollama pull gemma3:12b \
    && ollama pull ministral-3:8b && ollama pull cogito:14b
python run_gold_judges.py --smoke # connectivity check
python run_gold_judges.py # full run (~60 min)
```

Run the notebook: open `Assignment_2_BPClassifier.ipynb` in Jupyter and run cells top-to-bottom. All expensive steps (sentence extraction, embeddings, judge labels, FinBERT weights) are cached — re-runs skip completed steps automatically.

Run the GUI:

```
/Users/yueqilin/anaconda3/bin/python -m streamlit run gui.py
```